Assignment Name: Project 2: Optimizing GEMM for Deep Learning

Class Name and Section: ECE452-001	**Date Submitted:** 04/14/2026
Student Name: Arnav Revankar	[Github Link for Code]()

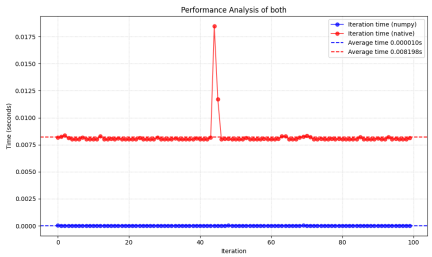
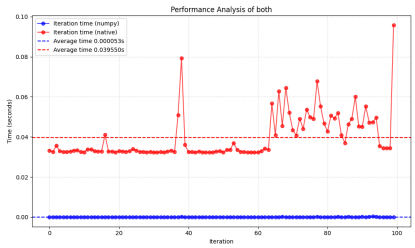
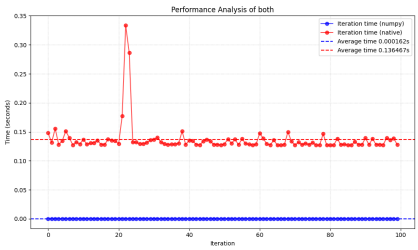
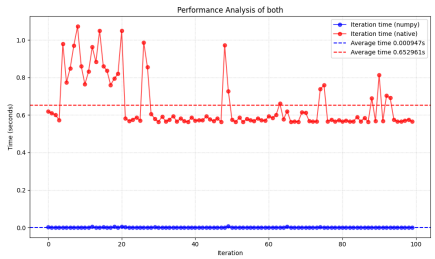
Introduction

In this project, I used both Numpy, and Native python, to implement different matrix multiplication algorithms for the purposes of benchmarking. Namely, I implemented the standard O(n^3) matrix multiplication method, and a cache-aware, tiling method. I then used Numpy to implement the same matrix multiplication on an Nvidia Tesla T4 GPU, from Google Colab. In this report, I'll discuss the differences that I found.

CPU - Naive

First off, I implemented a baseline, naive implementation using native Python, and Numpy. I used numpy because the cache-optimizations that this experiment tries to show with the tiling method will be nullified by Python's slow interpreter. Numpy, on the other hand, has a C backend, and is relatively fast. This shown in the initial simulations.

Below are four such instances of matrix MAC operations. This involved multiplying an NxN matrix by a Nx1 vector, and adding a 2nd Nx1 vector, for N = 128, 256, 512, and 1024. Not at all surprisingly, Numpy outperforms the traditional python each and every time. Numpy, being a C library, is naturally very fast. Python on the other hand seems to run between 100x to 1000x slower than Numpy, with occasional spikes, which I can only assume are caused by either an internal-interpreter delay, or OS preemption.

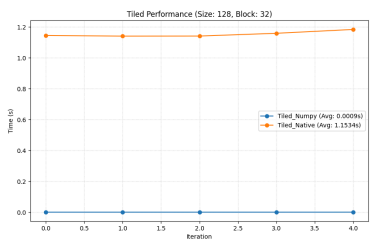
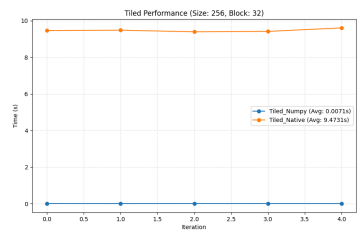
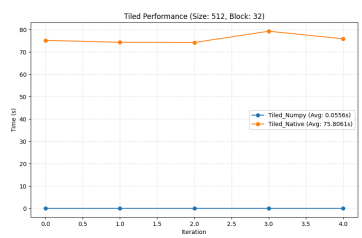
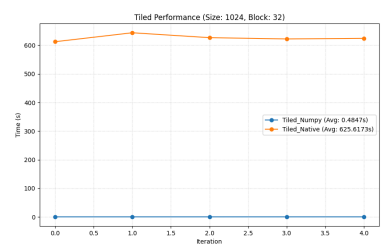
N = 128	**N = 256**
	
N = 512	**N = 1024**
	

CPU - Tiled

For the next part, I implemented the same thing as before but with a *tiled* matrix multiplication, with configurable block size. For the native python version, this consisted of 6 nested loops instead of just 3, to iterate over and sum all the individual tiles. Likewise, for the numpy version, this consisted of narrowing down the original matrix multiplication line to multiply and accumulate just one tile, and then wrapping it in the 3 surrounding for loops. The batch size for this tiled version and the GPU were both 32.

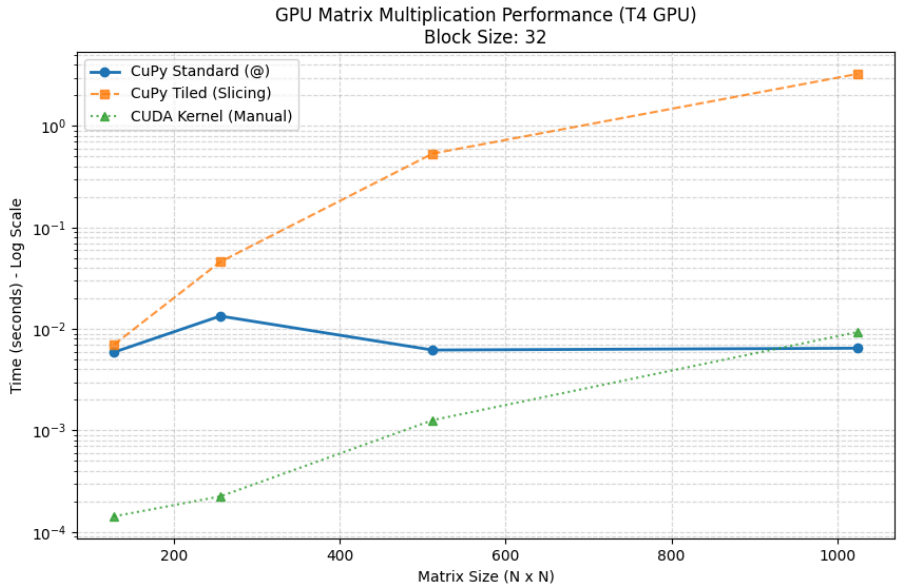
As expected, the python implementation did much, much worse. The average multiplication times for the 4 matrix sizes went up from 0.0082, 0.0396, 0.136, and 0.652 seconds respectively, to 1.15, 9.47, 75.81, and 626 seconds respectively. Being an interpreted language (that too, interpreted at runtime, unlike something like Java), Python lacks robustness when it comes to complex, iterative procedures. In these cases, it's usually much better to either use a compiled language, or something like Numpy.

However, unexpectedly, the numpy version *also* did poorly. Namely, it ran at 0.0009, 0.0071, 0.0556, and 0.485 seconds respectively. I believe the cause of this is the *added overhead of the 3 nested loops*. The 3 high level nested loops are *python* loops, and thus add more overhead than a C loop. Below are the results.

N = 128	**N = 256**
	
N = 512	**N = 1024**
	

GPU

For the GPU version, I used CuPy, which is designed as a drop in replacement for Numpy. I also included an inline-C version, which directly interacts with the CUDA kernel, through CuPy. I then ran this on a Google Colab instance, using an Nvidia Tesla T4 GPU. The results are shown in the plot below.



As you can see, the fastest by far for small matrices is the inline-C, which I assume bypasses the initially sluggish Python by directly interacting with the cuda kernel. However, with larger matrices, that initial sluggishness wears off as both the inline-C and standard CuPy run the actual multiplication at the same speed on the GPU. Finally, the tiled CuPy version (same as for numpy, since CuPy is a drop-in replacement) yet again underperformed both standard multiplications. Yet again, I believe this is because of the inherent python overhead, when running the 3 nested loops. The only way to bypass this is to write this same benchmark in a lower level language such as C, C++, Go, etc, which is *compiled* into native machine code rather than interpreted. Arguably even Java would outperform Python in this application, as the bytecode is generated beforehand, and only run on the VM at runtime.

Interestingly, for small matrix sizes, the GPU implementation *falls short of even the Naive CPU implementation*. In the GPU implementation, the fastest run with N=128 yielded a time of 0.00015s, whereas on the CPU (an old 6th gen Core-i7) the same ran in 0.00001s. Truth be told, I do not know the reason for this; my best guess would be due to latency of moving weights from main memory to the GPU's RAM, and also due to the kernel's initialization time.

Conclusion

In conclusion, we found that in all 3 cases (CPU Naive, CPU Tiled, GPU Tiled), the ultimate winner is the Naive CPU for small matrix sizes (<256), and the Tiled GPU for larger matrix sizes. In all likelihood, the Tiled CPU must also be relatively fast, and perhaps faster than the Naive CPU for large matrix sizes, yet I could not find that out as the sluggish python interpreter got in the way. Future tests to run before reaching a solid conclusion would be to recreate the Naive and Tiled CPU tests in a lower level language, to understand the actual performances.